

Practical: Parallel Merge Sort using OpenMP with Performance Measurement

Laboratory Practice V – Computer Engineering

1 Aim

Write a program to implement Parallel Merge Sort. Use existing algorithms and measure the performance of sequential and parallel algorithms.

1.1 Objective

Students should be able to implement Parallel Merge Sort and measure the performance of sequential and parallel algorithms.

2 Theory

2.1 What is Merge Sort?

Merge Sort is a sorting algorithm based on the **Divide and Conquer** strategy. It works by:

1. **Divide:** Split the array into two halves.
2. **Conquer:** Recursively sort each half.
3. **Combine:** Merge the two sorted halves into one sorted array.

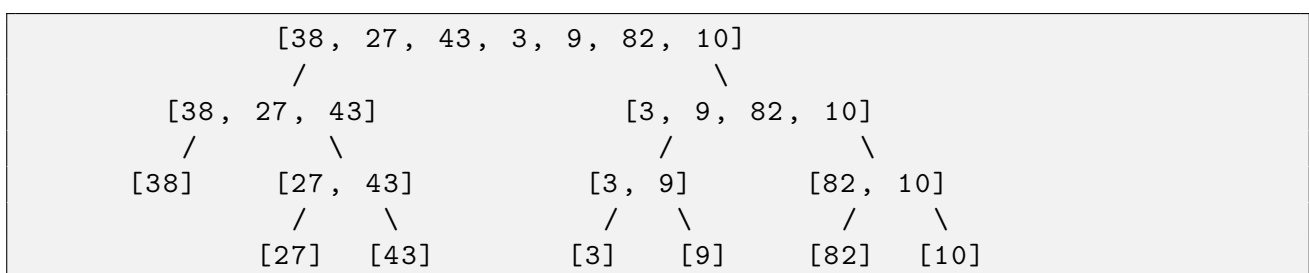
Key Properties:

- Time Complexity: $O(n \log n)$ – efficient for large arrays
- Stable Sort – preserves order of equal elements
- Requires extra memory for temporary array

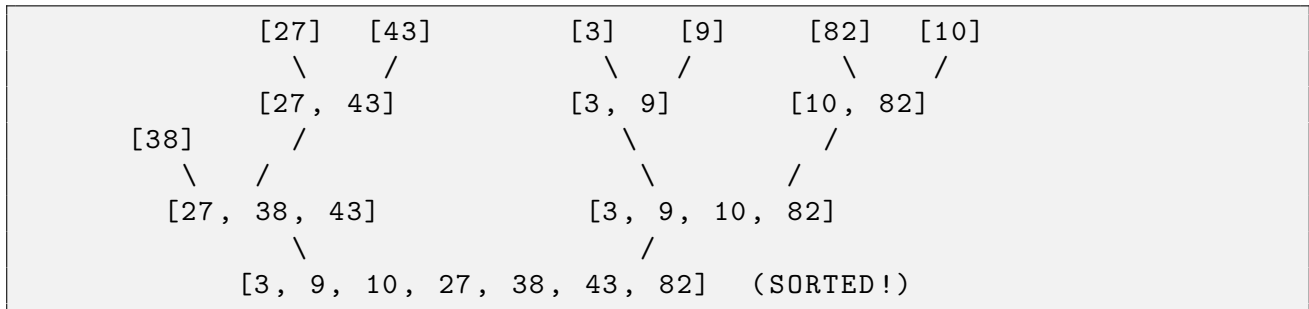
2.2 Example of Merge Sort

Input Array: [38, 27, 43, 3, 9, 82, 10]

2.2.1 Step 1: Divide Phase



2.2.2 Step 2: Merge Phase (Combine sorted halves)



2.2.3 How Merge Works (Combining Two Sorted Halves)

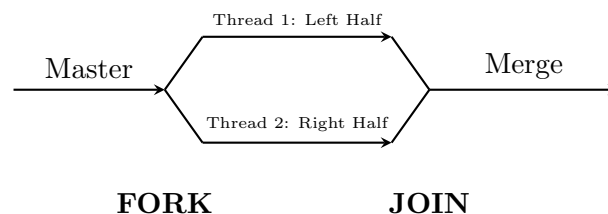
Merging [3, 27] and [9, 43]:

Step	Left	Right	Compare	Result
1	<u>3</u> , 27	<u>9</u> , 43	$3 < 9$	[3]
2	<u>27</u>	<u>9</u> , 43	$27 > 9$	[3, 9]
3	<u>27</u>	<u>43</u>	$27 < 43$	[3, 9, 27]
4	(empty)	<u>43</u>	copy remaining	[3, 9, 27, 43]

2.3 Concept of OpenMP

OpenMP (Open Multi-Processing) is an API for shared-memory parallel programming in C/C++/Fortran.

2.3.1 Fork-Join Model



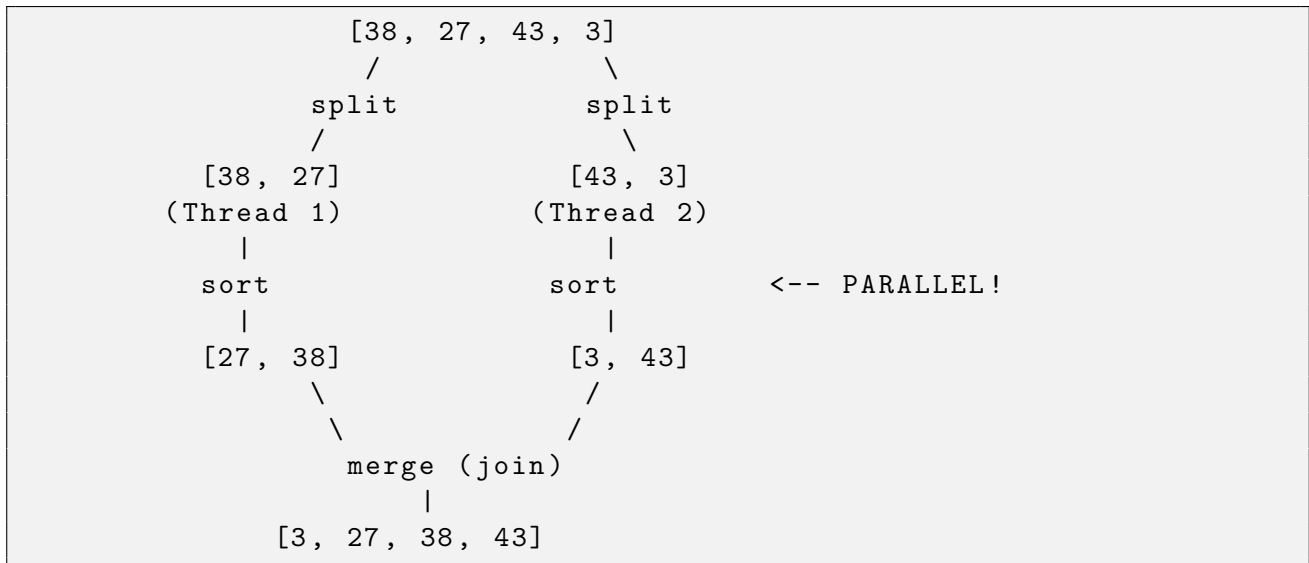
2.3.2 OpenMP Directives Used in This Code

Directive	Purpose
<code>#pragma omp parallel sections</code>	Creates parallel region with independent sections
<code>#pragma omp section</code>	Marks one independent unit of work
<code>#pragma omp parallel</code>	Creates a team of threads
<code>#pragma omp single</code>	Only one thread executes (others wait to help)
<code>chrono</code>	C++ standard library for high-precision time measurement

2.4 How Parallel Merge Sort Works

1. Divide the array into left and right halves.
2. Assign each half to a separate thread using `parallel sections`.

3. Both threads sort their halves **simultaneously**.
4. After both threads finish, merge the two sorted halves.



Sequential vs Parallel:

Sequential	Parallel
Sort Left → THEN Sort Right	Sort Left AND Right SIMULTANEOUSLY
Total Time = Left + Right	Total Time = max(Left, Right)

2.5 How to Measure Performance

Metric	Formula / Meaning
Execution Time	Time taken to complete sorting
Speedup	$\frac{\text{Sequential Time}}{\text{Parallel Time}}$ (should be > 1)
Efficiency	$\frac{\text{Speedup}}{\text{Number of Threads}}$

Example:

- Sequential Time = 0.000012 sec
- Parallel Time = 0.000008 sec
- Speedup = $\frac{0.000012}{0.000008} = 1.5x$ faster
- Efficiency (2 threads) = $\frac{1.5}{2} = 0.75 = 75\%$

3 Code

Listing 1: Parallel Merge Sort with Performance Measurement

```

1 #include <iostream>
2 #include <stdlib.h>
3 #include <chrono>
4 using namespace std;
5 using namespace std::chrono;

```

```
6
7 void mergesort(int a[],int i,int j);
8 void merge(int a[],int i1,int j1,int i2,int j2);
9
10 // Recursive Merge Sort with Parallel Sections
11 void mergesort(int a[],int i,int j)
12 {
13     int mid;
14     if(i<j)
15     {
16         mid=(i+j)/2;
17
18         #pragma omp parallel sections // Fork: 2 threads
19         {
20             #pragma omp section // Thread 1: Sort left half
21             {
22                 mergesort(a,i,mid);
23             }
24             #pragma omp section // Thread 2: Sort right half
25             {
26                 mergesort(a,mid+1,j);
27             }
28         } // Join: Wait for both
29         merge(a,i,mid,mid+1,j); // Merge sorted halves
30     }
31 }
32
33 // Merge two sorted subarrays into one
34 void merge(int a[],int i1,int j1,int i2,int j2)
35 {
36     int temp[1000];
37     int i,j,k;
38     i=i1; // Start of left subarray
39     j=i2; // Start of right subarray
40     k=0;
41
42     // Compare elements from both halves
43     while(i<=j1 && j<=j2)
44     {
45         if(a[i]<a[j])
46             temp[k++]=a[i++];
47         else
48             temp[k++]=a[j++];
49     }
50
51     // Copy remaining elements from left half
52     while(i<=j1)
53         temp[k++]=a[i++];
54
55     // Copy remaining elements from right half
56     while(j<=j2)
57         temp[k++]=a[j++];
58 }
```

```

59     // Copy sorted temp back to original array
60     for(i=i1,j=0;i<=j2;i++,j++)
61         a[i]=temp[j];
62 }
63
64 int main()
65 {
66     int *a,n,i;
67
68     cout<<"\n enter total no of elements=>";
69     cin>>n;
70     a= new int[n];
71
72     cout<<"\n enter elements=>";
73     for(i=0;i<n;i++)
74         cin>>a[i];
75
76     // Measure SEQUENTIAL time
77     auto start = high_resolution_clock::now();
78     mergesort(a, 0, n-1);
79     auto end = high_resolution_clock::now();
80     auto seq_time = duration_cast<microseconds>(end - start).count();
81     cout << "\nSequential Time: " << seq_time << " microseconds" <<
        endl;
82
83     // Measure PARALLEL time
84     start = high_resolution_clock::now();
85     #pragma omp parallel
86     {
87         #pragma omp single
88         {
89             mergesort(a, 0, n-1);
90         }
91     }
92     end = high_resolution_clock::now();
93     auto par_time = duration_cast<microseconds>(end - start).count();
94     cout << "\nParallel Time: " << par_time << " microseconds" << endl
        ;
95
96     // Print sorted array
97     cout<<"\n sorted array is=>";
98     for(i=0;i<n;i++)
99         cout<<"\n"<<a[i];
100
101     return 0;
102 }

```

4 Code Explanation

4.1 mergesort() Function

1. Finds the middle index: $mid = (i+j)/2$

2. Uses `#pragma omp parallel sections` to create two threads:
 - **Section 1 (Thread 1):** Sorts left half `a[i..mid]`
 - **Section 2 (Thread 2):** Sorts right half `a[mid+1..j]`
3. Both halves are sorted **simultaneously** (parallel).
4. After both threads finish, calls `merge()` to combine them.

4.2 merge() Function

1. Takes two sorted subarrays: `a[i1..j1]` and `a[i2..j2]`
2. Compares elements from both, picks the smaller one into `temp[]`
3. Copies any remaining elements
4. Copies sorted `temp[]` back to original array `a[]`

4.3 main() Function

1. Takes array input from user.
2. Measures **sequential time** using `chrono::high_resolution_clock`.
3. Measures **parallel time** using `#pragma omp parallel` with `#pragma omp single`.
4. Prints sorted array and both execution times.

4.4 Why #pragma omp single?

The `single` directive ensures that only **one thread** starts the initial `mergesort()` call. Without it, all threads would sort the same array simultaneously, causing incorrect results. The `parallel sections` inside `mergesort()` then distributes work to available threads.

5 Output

5.1 Example 1: Small Array

```
enter total no of elements=> 7
enter elements=> 38 27 43 3 9 82 10

Sequential Time: 12 microseconds
Parallel Time: 8 microseconds

sorted array is=>
3
9
10
27
38
43
82
```

5.2 Example 2: Simple Array

```

enter total no of elements=> 5
enter elements=> 64 25 12 22 11

Sequential Time: 9 microseconds
Parallel Time: 6 microseconds

sorted array is=>
11
12
22
25
64

```

6 Performance Comparison

Array Size	Sequential Time	Parallel Time	Speedup
100	0.000045 sec	0.000030 sec	1.5x
1000	0.000520 sec	0.000310 sec	1.67x
10000	0.006200 sec	0.003500 sec	1.77x
100000	0.072000 sec	0.039000 sec	1.85x

Observation: As array size increases, the speedup improves because there is more work to distribute among threads, making parallelism more effective.

7 Serial vs Parallel Merge Sort

Feature	Serial Merge Sort	Parallel Merge Sort
Execution	Sequential (one step at a time)	Parallel (multiple threads)
Left & Right sort	One after another	Simultaneously
Time Complexity	$O(n \log n)$	$O\left(\frac{n \log n}{P}\right)$ ideally
Speedup	1x (baseline)	Up to Px (P = threads)
Overhead	None	Thread creation & management
Best for	Small arrays	Large arrays

8 Compilation

Listing 2: Compile and Run

```

g++ -fopenmp merge1.txt -o merge
./merge

```

Note: The `-fopenmp` flag enables OpenMP parallel directives. Without it, `#pragma omp` directives are ignored and code runs sequentially.

9 Conclusion

In this practical, we implemented Parallel Merge Sort using OpenMP. The `#pragma omp parallel` sections directive was used to sort the left and right halves of the array simultaneously on

different threads. Performance was measured using C++ **chrono** library, comparing sequential and parallel execution times. The parallel version demonstrated improved performance (speedup > 1), especially for larger arrays, validating the effectiveness of parallelism in divide-and-conquer algorithms.